- *Databricks in general*—Databricks is a comprehensive analytics platform that can run on every major cloud. We're using only a part of the platform offering here.
- *Databricks Delta Lake*—This smart object storage format used by Databricks provides a bunch of database-like guarantees for your analytical workloads.
- *Apache Spark*—This is a multilanguage framework for executing analytical workloads.
- *Databricks notebook*s—These are notebooks you can use to write Python- or Scala-based Spark code. This can be scheduled as a batch or streaming job.
- *Databricks SQL*—This is basically a wrapper around the BI tool Redash, well integrated into the Databricks landscape.

Comparing this architecture to the AWS and GCP-based architectures, you should be able to notice the following:

- This architecture is very intertwined in terms of the technology different teams use. The teams share Delta Lake, so separate (folder-based) boundaries have to be put in to mark the domains.
- But in turn, it is easy to set up and maintain.
- Most technology choices work inside the Databricks context.
- We need a few additional technologies (here, AWS based) to get the same scope as we had in the other data meshes. These concern getting data into the centralized data structures.
- A lot of out-of-the-box technologies assist analytical working, like notebooks and Databricks SQL for graphical analysis. Even the recommendation engine can be built inside Databricks.
- This architecture, unlike the previous two, doesn't utilize infrastructure provisioning templates.
- This architecture doesn't utilize a data catalog. A Databricks internal data catalog can be used, but *data catalog* in the data mesh context usually refers to technologies aiming to cover physically distributed data pieces. Since this is not the case here, the platform can work without it for now. See section 8.3.5 for more details.

Let's again identify the components of a data mesh: first the components of the platform and then the data product part.